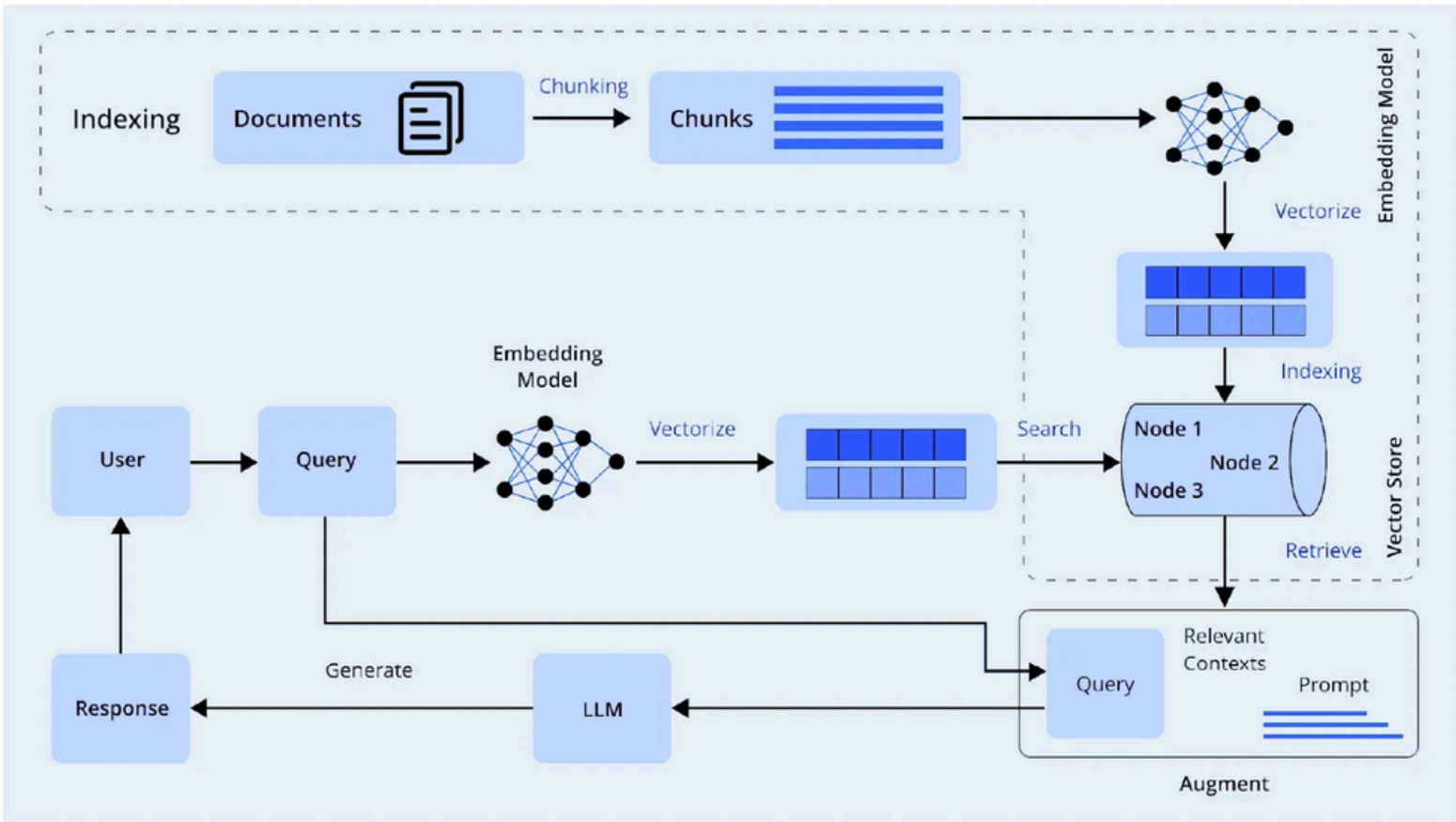
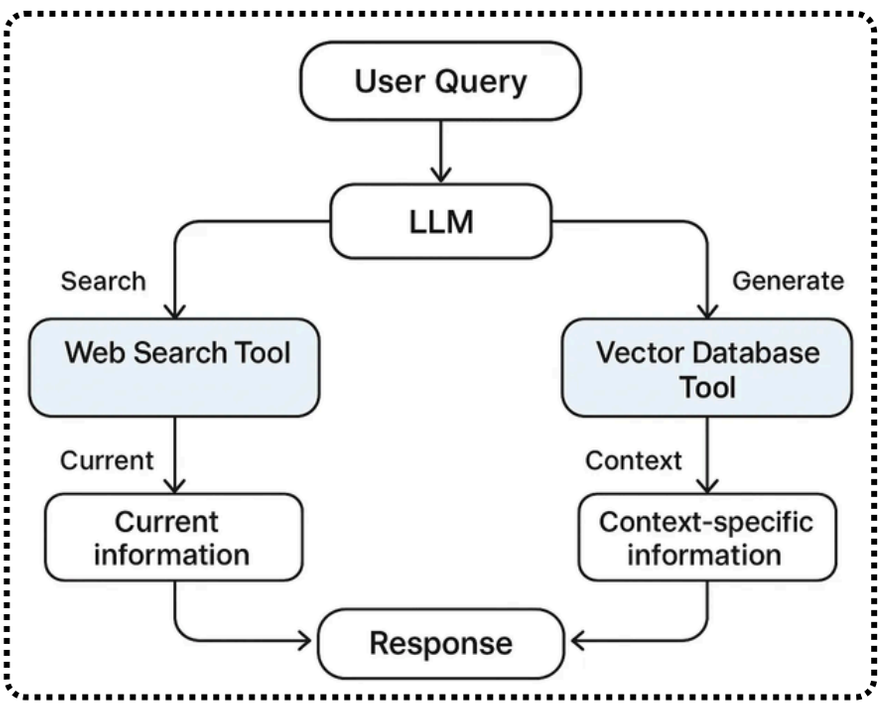
Analytics Vidhya

# RAG for Multi-Tool Integration and Smart Workflows




**Swipe Left to know more...**

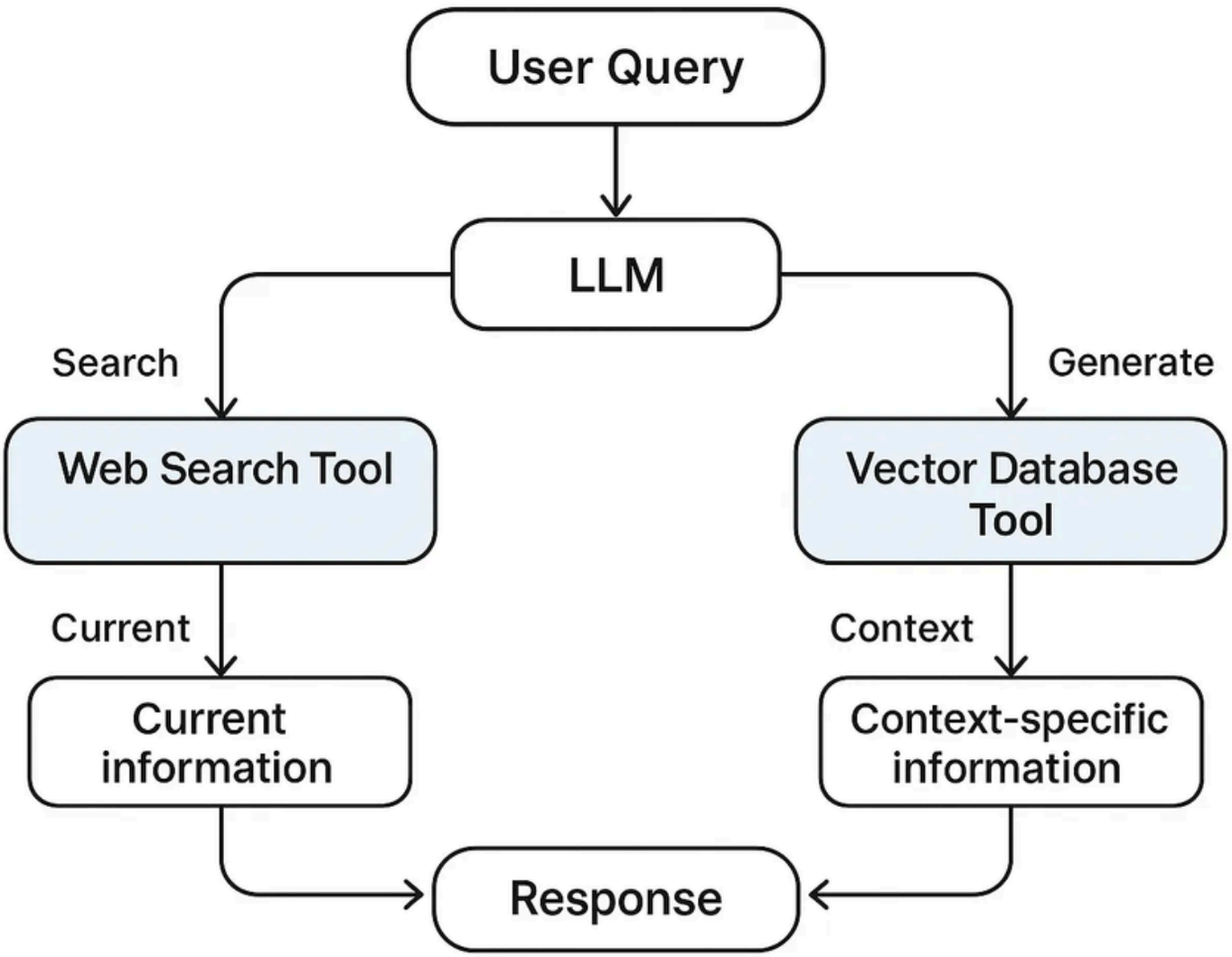
Multi-Tool Orchestration with Retrieval-Augmented Generation (RAG) is about creating intelligent workflows that employ large language models (LLMs) with tools, including web search engines or vector databases, to respond to queries.

By doing so, the LLM will automatically and dynamically select which tool to use for each query. For example, the web search tool will open the domain of current updated information, and the vector database, like Pinecone, for the context-specific information.

In practice, RAG often entails defining function-call tools, such as web search or database lookup, and orchestrating these via an API, e.g., the Responses API or OpenAI. This use initiates a sequence of retrieval and generation steps for each user query. As a result, aspects of the model's ability are intertwined with current information.

# Creating Multi-Tool Orchestration with RAG

Now, we'll go through a real-world example of creating a multi-tool RAG system using a medical Q&A dataset. The process is, we will embed a question-answer dataset into Pinecone and set up a system. The model has a web-search tool and a pinecone-based search tool. Here are some steps and code samples from this process.



# Loading Dependencies and Datasets

First, we'll install, then import the necessary libraries, and lastly download the dataset. It will require a basic understanding of data handling, embeddings, and the Pinecone SDK. For example:

```
import os, time, random, string

import pandas as pd

from tqdm.auto import tqdm

from sentence_transformers import SentenceTransformer

from pinecone import Pinecone, ServerlessSpec

import openai

from openai import OpenAI

import kagglehub
```

Next, we will download and load a dataset of medical questions and answer relationships. In the code, we used the Kagglehub utility to access a medically focused QA dataset:

```
path = kagglehub.dataset_download("thedevastator/comprehensive-
medical-q-a-dataset")

DATASET_PATH = path  # local path to downloaded files

df = pd.read_csv(f"{DATASET_PATH}/train.csv")
```

For this example version, we can take a subset, i.e., the first 2500 rows. Next, we will prefix the columns with “Question:” and “Answer:” and merge them into one text string. This will be the context we will embed. We are making embeddings out of text. For example:

```
df = df[:2500]

df['Question'] = 'Question: ' + df['Question']

df['Answer'] = ' Answer: ' + df['Answer']

df['merged_text'] = df['Question'] + df['Answer']
```

The merged text from rows looked like: “Question: [medical question] Answer: [the answer]”

**Question: Who is at risk for Lymphocytic Choriomeningitis (LCM)?**

**Answer**: LCMV infections can occur after exposure to fresh urine, droppings, saliva, or nesting materials from infected rodents. Transmission may also occur when these materials are directly introduced into broken skin, the nose, the eyes, or the mouth, or presumably, via the bite of an infected rodent. Person-to-person transmission has not been reported, except vertical transmission from infected mother to fetus, and rarely, through organ transplantation.’

# Creating the Pinecone Index Based on the Dataset

Now that the dataset is loaded, we will produce the vector embedding for each of the merged QA strings. We will use a sentence-transformer model “BAAI/bge-small-en” to encode the texts:

```
MODEL = SentenceTransformer("BAAI/bge-small-en")

embeddings = MODEL.encode(df['merged_text'].tolist(),
show_progress_bar=True)

df['embedding'] = list(embeddings)
```

We will take the embedding dimensionality from a single sample ‘len(embeddings[0]’. For our case, it is 384. We will then create a new Pinecone index and give the dimensionality. This is done using the Pinecone Python client:

```python
def upsert_to_pinecone(df, embed_dim, model, api_key, region="us-east-1", batch_size=32):
    # Initialize Pinecone and create the index if it doesn't exist
    pinecone = Pinecone(api_key=api_key)
    spec = ServerlessSpec(cloud="aws", region=region)
    index_name = 'pinecone-index-' + ''.join(random.choices(string.ascii_lowercase + string.digits, k=10))
    if index_name not in pinecone.list_indexes().names():
        pinecone.create_index(
            index_name=index_name,
            dimension=embed_dim,
            metric='dotproduct',
            spec=spec
        )
    # Connect to index
    index = pinecone.Index(index_name)
    time.sleep(2)
    print("Index stats:", index.describe_index_stats())
    # Upsert in batches
    for i in tqdm(range(0, len(df), batch_size), desc="Upserting to Pinecone"):
        i_end = min(i + batch_size, len(df))
        # Prepare input and metadata
        lines_batch = df['merged_text'].iloc[i:i_end].tolist()
        ids_batch = [str(n) for n in range(i, i_end)]
        embeds = model.encode(lines_batch, show_progress_bar=False, convert_to_numpy=True)
        meta = [
            {
                "Question": record.get("Question", ""),
                "Answer": record.get("Response", "")
            }
            for record in df.iloc[i:i_end].to_dict("records")
        ]
```

Here, each vector is being stored with its text and metadata. The Pinecone index is now populated with our domain-specific dataset.

## Query the Pinecone Index

To use the index, we define a function that we can call the index with a new question. The function embeds the query text and calls index.query to return the top-k most similar documents:

```
def query_pinecone_index(index, model, query_text):

    query_embedding = model.encode(query_text, convert_to_numpy=True).tolist()
    res = index.query(vector=query_embedding, top_k=5, include_metadata=True)

    print("--- Query Results ---")

    for match in res['matches']:
        question = match['metadata'].get("Question", 'N/A')
        answer = match['metadata'].get("Answer", "N/A")
        print(f"{match['score']:.2f}: {question} - {answer}")

    return res
```

As an example, if we were to call query_pinecone_index(index, MODEL, "What is the most common treatment for diabetes?"), we will see the top matching Q&A pairs from our dataset printed out. This is the retrieval portion of the process: the user query gets embedded, looks up the index, and returns the closest documents (as well as their metadata). Once we have those contexts retrieved, we can use them to help formulate the final answer.